UNIVERSIDAD CAECE

Cámara Argentina de Comercio y Servicios

LICENCIATURA EN CIENCIAS DE DATOS

ESTRUCTURAS Y ALGORITMOS DE DATOS II

GUIA DE LA UNIDAD Nro. 3

Contenidista: Maximiliano López

ÍNDICE

1- INTRODUCCIÓN 2

2- DESARROLLO 3

3- CONCLUSIONES 25

4- BIBLIOGRAFÍA DE REFERENCIA 25

FIGURAS:

Ilustración 1 comer un paquete de galletas ITERATIVO 4
Ilustración 2 comer un paquete de galletas RCURSIVO 5
Ilustración 3 ejecución paso a paso 6
Ilustración 4Definición de factorial 9
Ilustración 5 Función Factorial 10
Ilustración 6 Factorial paso a paso 10
Ilustración 7 Definición de FIBONACCI 12
Ilustración 8 Función FIBONACCI 12
Ilustración 9 Función SumaNat 13
Ilustración 10 función Digitos 14
Ilustración 11 Función máximo de arreglo 15
Ilustración 12 Función suma de elementos de arreglo 16
Ilustración 13 función Existe 17
Ilustración 14 Quicksort paso a paso 20
Ilustración 15 Procedimiento Quicksort 21
Ilustración 16 Mergesort paso a paso 23
Ilustración 17 Procedimiento Mergesort 25

1- INTRODUCCIÓN

En la Unidad anterior le presentamos una estrategia para poder construir programas más complejos y luego poder verificarlos. Ahora, le propondremos otra forma de resolver problemas con la particularidad que el proceso mediante el cual se encuentra la solución utiliza su propia definición para lograrlo.

2- DESARROLLO

1.1 EL PROCESO AUTOREFERENCIAL

Como le adelantamos en la introducción, le presentaremos una nueva manera de encontrar la solución a un problema algorítmico que utilice rutinas, la Recursividad. Este proceso recursivo lo podemos ver como una forma de resolver problemas, con la particularidad de que el proceso mediante el cual hallamos la solución deseada utiliza su propia definición para lograrlo. Es decir que la recursividad es una Rutina que se invoca a sí misma.

LA RUTINA SE INVOCA A SÍ MISMA

1.2 ITERACIÓN VS RECURSIÓN

1.2.1 PROCESO ITERATIVO

Ahora le proponemos que analicemos el proceso con el cual usted ha trabajado en todas las materias de programación hasta ahora, es decir la manera Iterativa. Cuando usted resuelve un problema iterativamente, seguramente utilizará un índice inicializado en cierto valor, y lo irá modificando dentro de una estructura de control iterativa (podrá Mientras, Repetir o Para) hasta que se cumpla cierta condición con dicho índice, y entonces se termina el proceso iterativo.

Para ejemplificar este proceso con un caso de la vida diaria, pensemos que nos proponemos la tarea de comer un paquete de galletas. Desde un punto de vista iterativo, nosotros podríamos conceptualmente y a modo de ejemplo plantear un código como el siguiente:

```
// PRE: cant_galletas > 0
Procedimiento comer_paquete (val: cant_galletas entero)
    // supongamos que el paquete tiene 4 galletas, por lo que cant_galletas = 4
    VAR LOC: i: entero
    Para i ← 1 hasta cant_galletas hacer
       comer_galleta()
    fPara
    //cuando salimos del Para, se ejecutó comer_galleta() 4 veces
fProced
```

Ilustración 1 comer un paquete de galletas ITERATIVO

Según este algoritmo, podrá ver que vamos explícitamente comiendo todas las galletas, una a una, hasta que ya no queden más en el paquete (es decir hasta que i llegue a cant_galletas). Habrá notado, seguramente, que esta forma de proceder es coherente con el modo en que resuelve problemas, es decir, en forma iterativa.

1.2.2 PROCESO RECURSIVO

Ahora, vamos a ver cómo podemos resolver esta misma problemática desde otro punto de vista que es, a los efectos finales, equivalente al proceso iterativo que le explicamos en la sección anterior, pero con ciertas diferencias conceptuales importantes.

La clave es entender que, ahora, "comer un paquete de galletas" no es ir comiendo explícitamente todas las galletas dentro de un bucle como hicimos primero, sino que bajo esta nueva perspectiva es equivalente a "comer una galleta, más comer otro paquete de galletas", donde el "otro paquete" es más pequeño. A su vez, comer el paquete de galletas más pequeño implica, nuevamente, comer otra galleta, y a su vez comer otro paquete de galletas, pero más pequeño aún (siempre estamos repitiendo la misma idea una y otra vez, de forma autorreferencial). Tanto el paquete original como los sub paquetes, los vamos tratando de la misma forma: comiendo una galleta y comiendo otro paquete más pequeño.

Este mismo proceso se debe repetir una y otra vez hasta que llegamos a un punto en el que el "otro paquete" está vacío, y ya no hay más galletas para comer.

De esta manera, podemos plantear el siguiente algoritmo de forma recursiva.

```
//PRE: cant_galletas > 0 en la primera invocación
Procedimiento comer_paquete(val cant_galletas: entero)
   // supongamos que el paquete tiene 4 galletas, por lo que cant_galletas = 4
   Si cant_galletas >= 1 entonces
      comer_galleta()
      comer_paquete(cant_galletas-1) /* el procedimiento se llama a sí mismo
                                        pero decrementando el parámetro en 1 */
   fSi
fProced
```

Ilustración 2 comer un paquete de galletas RCURSIVO

Observe con más atención el comportamiento del parámetro cant_galletas. Mientras el parámetro sea mayor o igual a 1, procederemos a "comer una galleta y comer otro paquete", indicando un paquete más pequeño mediante el parámetro cant_galletas - 1.

En algún momento, se llamará a comer_paquete(0), y entonces al invocarse a sí mismo, encontraremos que no es cierto que 0 >= 1, y no podremos comer una galleta y comer otro paquete, porque habremos llegado a un paquete vacío. En tal caso, habrán terminado los llamados recursivos, y nos encontraremos con que habremos comido todas las galletas del paquete original.

Es importante destacar que con este algoritmo se está descomponiendo el problema principal y llevándolo a instancias más chicas de sí mismo que podemos manejar.

Una representación paso a paso de la ejecución de este código podría ser la siguiente:

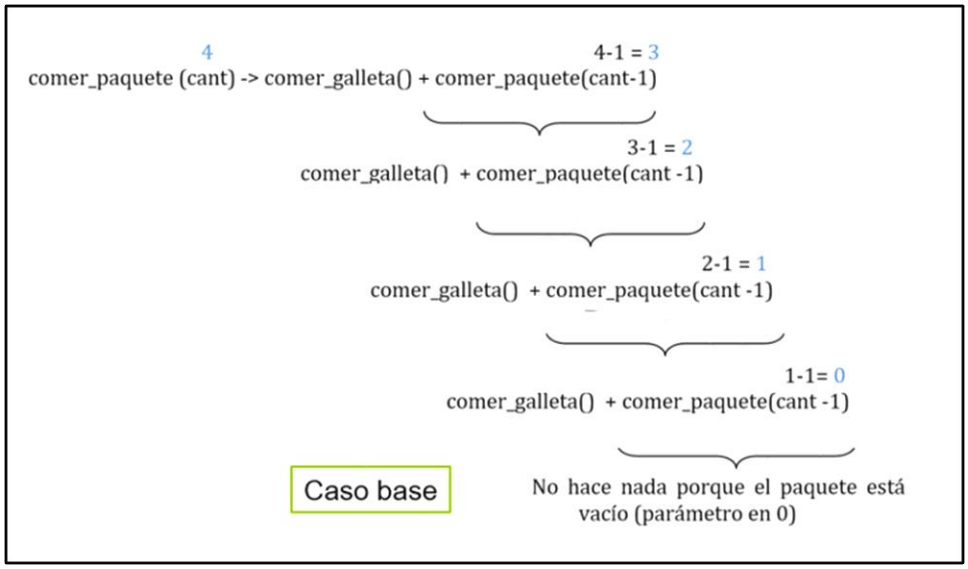

Ilustración 3 ejecución paso a paso

Lo primero que tendrá que observar es que la definición del problema implica una autorreferencia a la definición del problema en sí mismo, que es comer un paquete de galletas. Esto es así porque, "comer un paquete de galletas" equivale a "comer 1 galleta y comer otro paquete de galletas".

Esta idea que le destacamos es la clave del proceso autorreferencial. Se está tomando el problema original ("comer un paquete de galletas") y subdividiéndolo en versiones

más pequeñas del mismo problema que sean más manejables ("comer 1 galleta y comer otro paquete de galletas, ahora más pequeño").

A este proceso se lo conoce más comúnmente como divide y vencerás.

Esta estrategia es muy similar a la que le presentamos cuando repasamos el concepto de Rutinas en la Unidad anterior. Así es como se irá subdividiendo el problema en instancias más pequeñas del mismo problema, hasta llegar a un punto en el que nos daremos cuenta de que ya no podemos seguir comiendo una galleta más otro paquete más pequeño. A este momento en el que llegamos a un punto irreducible (ya no podemos subdividir al paquete de galletas en paquetes más pequeños), lo vamos a llamar caso base. Es el momento en el que el proceso autorreferencial termina.

En ese momento, en el que se finalizan los llamados recursivos, si usted observa para atrás, notará que se ha resuelto el problema de comer el paquete de galletas, reduciéndolo a una simplificación de sí mismo, comiendo de a una galleta hasta llegar a la situación en la que el paquete ha quedado vacío.

Observe también que comer_galleta() podría haberse invocado justo antes o después del llamado recursivo a comer_paquete(), y el algoritmo habría cumplido su objetivo por igual. De todas formas, le aclaramos que, en términos prácticos, ambas opciones son distintas y que, si bien en este curso se aborda una introducción a la temática de la recursividad, hay múltiples situaciones y áreas de estudio en donde este orden de invocación sí es importante para encontrar mejores soluciones a otras (involucrando, por ejemplo, conceptos tales como recursión por izquierda o recursión por derecha, entre otras clasificaciones), en las cuales no nos adentraremos.

1.2.2.1 PASOS DE UN ALGORITMO RECURSIVO

Seguramente se habrá dado cuenta de que trabajar de manera recursiva implica otra forma de pensar y de resolver el problema. Estará pensando, también, cómo plantear un algoritmo recursivo, y para eso le contaremos los pasos que deberá realizar para poder desarrollarlo.

1. Identificar el o los casos base (sabemos cómo resolverlos en forma concreta)
2. Descomponer el problema principal en versiones más pequeñas y simples de sí mismo hasta alcanzar el o los casos base
3. Encadenar o combinar los subproblemas que ya sabemos resolver para construir el resultado final

Ahora veamos estos pasos aplicados al ejemplo con el que venimos trabajando.

1. En el ejemplo anterior, el caso base es el paquete de galletas vacío (cant_galletas = 0), ya que sabemos resolver el caso de un paquete vacío: no hay nada para comer. De hecho, cuando comer_paquete() es invocado con el parámetro cant_galletas = 0, entonces cant_galletas no es >= 1, y el código no hace más nada (no hay más llamados recursivos).
2. La reducción del problema estuvo dada por ir tomando paquetes cada vez más pequeños mediante la invocación recursiva del procedimiento comer_paquete() con su parámetro cant_galletas decrementado en 1 cada vez que se lo llamó recursivamente, hasta que se llegue al punto en que su parámetro cant_galletas valga 0 (caso base).
3. Finalmente, como sabemos resolver el caso base (no hacer nada cuando el paquete esté vacío) y cómo ir descomponiendo el problema en versiones más chicas de sí mismo, agregamos comer_galleta() para cada paquete de galletas que vamos procesando, y de esa forma nos queda explícitamente que "comer un paquete de galletas implica comer una galleta y comer otro paquete de galletas, ahora más pequeño".

1.3 EJEMPLOS DE RECURSIVIDAD

En esta sección le proponemos pasar a la práctica y comenzar a desarrollar de manera recursiva nuestras primeras rutinas.

1.3.1 FACTORIAL

Recordemos la definición matemática de un número Factorial.

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n*(n-1)! & \text{si } n > 0 \end{cases}$$

Ilustración 4Definición de factorial

Tomemos como ejemplo cómo resolver: 5!

Podemos calcular

5! = 5*4! = 5*4*3! = 5*4*3*2! = 5*4*3*2*1! = 5*4*3*2*1*0! = 5*4*3*2*1*1 (ya que 0! = 1)

5! = 120

Ahora le proponemos pensar este algoritmo como el desarrollo de una función recursiva llamada fact(n). El caso base está explícitamente definido por la definición misma de la función: cuando n=0 entonces fact(n) debe devolver 1 (no habrá llamados recursivos a fact()). La descomposición del problema principal fact(n) involucra tener que usar fact(n-1) para ir reduciendo el problema hasta que lleguemos a fact(0) cuyo valor sabemos que será 1 por ser el caso base. Entonces, para construir el resultado final tendríamos que ir tomando cada fact(n-1) e ir multiplicándolo por n, tal como lo indica también la definición de factorial.

De esta manera, la función Recursiva quedará así.

```
1  //PRE: n >= 0
2  Función fact (val n: entero): entero
3     Si n > 0 entonces
4         fact ← n * fact(n-1)     // recursión
5     sino
6         fact ← 1         // caso base: recordar que 0! = 1
7     fSi
8  fFunc
```

Ilustración 5 Función Factorial

Veamos cómo sería el orden de las llamadas para el mismo ejemplo en el que queremos calcular: 5!

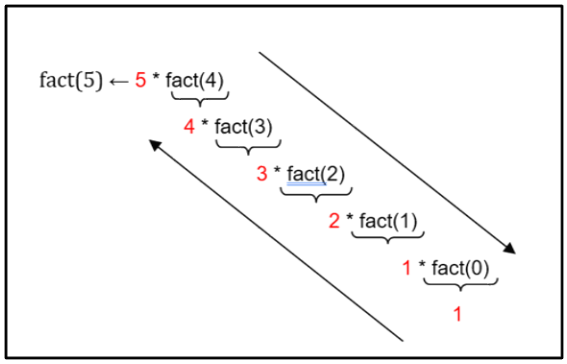

Ilustración 6 Factorial paso a paso

La primera vez que invocamos a esta función, le pasamos el parámetro 5 y así es como nos queda la llamada fact(5).

Inmediatamente, será cierto que 5 > 0 y por ende fact(5) devolverá 5 * fact(4), pero todavía falta resolver fact(4) para que fact(5) devuelva una respuesta concreta. Por ende, el contexto de fact(5) queda a la espera de que se resuelva fact(4) antes de poder

completar la totalidad del cálculo. Mientras tanto, el contexto de fact(5) se apila en la pila de ejecución, y se pasa la ejecución a fact(4), en donde a su vez será cierto que 4 > 0, por lo que fact(4) devolverá 4 * fact(3), el cual falta todavía resolverse, y así sucesivamente...

Entonces, se repite esto una y otra vez, hasta que se llega a un punto en el que hay que evaluar fact(0), y entonces llegaremos al caso base, puesto que es FALSO que 0 > 0, y entonces entramos en el sino. En ese momento fact(0) devuelve un valor concreto que es 1, ya que se trata de justamente el caso base de este algoritmo.

Llegado a este punto, ya no habrá más llamados recursivos, por lo que se empieza a construir el resultado final desapilando los contextos de las rutinas que se fueron apilando y de hecho haciendo que cada una pueda, ahora sí, devolver un valor concreto.

Así, fact(1) ← 1 * fact(0) equivale a decir 1 * 1, y entonces fact (2) ← 2 * 1 (1*1 = 1), y entonces fact(3) = 3 * fact (2) (2*1), y así sucesivamente. Cuando lleguemos a fact(5), nos encontraremos con que estaremos calculando 5*24, ya que 24 = 4*3*2*1*1, devolviendo como resultado 120, lo que corresponde a 5!

1.3.2 N-ésimo TÉRMINO DE FIBONACCI

La sucesión de fibonacci dice que, partiendo de dos números (0 y 1), se genera una sucesión de número estableciendo que el siguiente elemento de la sucesión equivale a sumar los dos términos anteriores. Por lo tanto, tendríamos:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

A cada uno de estos números se los llama "términos", en donde el término f0 es 0, f1 es 1, f2 es 1, f3 es 2, f4 es 3, f5 es 5, f6 es 8, y así sucesivamente.

Más formalmente, se puede decir:

$$f(n) = \begin{cases} f(n-1) + f(n-2) & \text{si } n >= 2 \\ 1 & \text{si } n = 1 \\ 0 & \text{si } n = 0 \end{cases}$$

Ilustración 7 Definición de FIBONACCI

Esta sucesión es recursiva por definición misma, y por ende podríamos crear una función recursiva que nos devolviera el n-ésimo término de Fibonacci:

```
//PRE: n >= 0
Función Fibo (val n: entero): entero
    Si n >= 2 entonces
        Fibo ← Fibo(n-1) + Fibo(n-2)
    sino
        Fibo ← n
    fSi
fFunc
```

Ilustración 8 Función FIBONACCI

1.3.3 SUMA DE N NÚMEROS NATURALES

En este ejemplo queremos realizar una Función que nos devuelva la suma de los primeros N naturales. Por ejemplo, si N = 5, la función devolverá:

SumaNat = 1+2+3+4+5 = 15

Este proceso lo podemos pensar de manera recursiva teniendo en cuenta que la suma de los números hasta N es igual a la suma de los números hasta N-1 + el mismo número N. Así, iremos dividiendo el problema hasta llegar al caso base cuando N = 1 (el primer número natural). Veamos cómo nos queda el algoritmo.

```
// PRE n >= 1
Funcion SumaNat (val n: entero): entero
    Si n > 1 entonces
        SumaNat ← n + SumaNat (n-1)
    sino
        SumaNat ← 1
    fSi
fFunc
```

Ilustración 9 Función SumaNat

1.3.4 CANTIDAD DE DÍGITOS DE UN NÚMERO ENTERO

Ahora le presentamos una Función que recibe un número entero por parámetro y devuelve la cantidad de dígitos que tiene ese número.

En este ejercicio vamos a realizar la operación DIV 10 para ir quitando cada uno de los dígitos del número. Utilizaremos la recursividad para pasar de un número al número menor, aprovechando cada llamada recursiva para contar el dígito que acabamos de sacar.

De este modo, nos queda la siguiente Función.

```
// PRE n >= 0
Funcion Digitos (val x: entero): entero
    Si x DIV 10 > 0 entonces
        Digitos ← 1 + Digitos(x DIV 10)
    sino
        Digitos ← 1
    fSi
fFunc
```

Ilustración 10 función Digitos

1.3.5 MÁXIMO ELEMENTO DE UN ARREGLO

La recursividad es muy práctica cuando se utilizan arreglos, ya que podemos pensar en el subproblema más pequeño, como si fuera el arreglo con un elemento menos.

En este ejercicio utilizaremos la llamada recursiva para obtener el máximo valor de un arreglo con un elemento menos, de este modo iremos reduciendo el problema (el arreglo con menos elementos) hasta llegar al caso base que se encuentra cuando el arreglo tiene 1 elemento, y en ese caso el máximo es el mismo elemento.

Vamos cómo es nuestro algoritmo.

```
1   / PRE {tope > 0}
2   Funcion max (ref x: arreglo[CMAX] de enteros, val tope: entero):entero
3       VAR LOC: maximo: entero
4       Si tope > 1 entonces
5           maximo ← max(x, tope -1)
6           Si x[tope] > maximo entonces
7               max ← x[tope]
8           sino
9               max ← maximo
10          fSi
11      sino
12          max ← x[1]
13      fSi
14  fFunc
```

Ilustración 11 Función máximo de arreglo

1.3.6 SUMA DE LOS ELEMENTOS DE UN ARREGLO

Esta rutina es sumamente práctica porque nos devuelve la suma de todos los elementos de un arreglo de enteros, seguramente la utilizará en muchos de sus programas.

Utilizando el mismo criterio del ejemplo 1.3.5 iremos pasando de modo recursivo a un arreglo con un elemento menos, hasta llegar al caso base cuando el arreglo tiene 1 solo elemento. En ese caso, se inicializa la suma con el mismo elemento.

```
// PRE {tope > 0}
Funcion Suma (ref x: arreglo[CMAX] de enteros, val tope: entero): entero
    Si tope > 1 entonces
        Suma ← x[tope] + Suma (x, tope-1)
    sino
        Suma ← x[1]
    fSi
fFunc
```

Ilustración 12 Función suma de elementos de arreglo

1.3.7 BUSCAR UN ELEMENTO EN UN ARREGLO

Este ejemplo es una función lógica que devuelve VERDADERO si el dato a buscar (parámetro elem) se encuentra en el arreglo. Por el contrario, devolverá FALSO cuando elem no se encuentre en el vector.

Iremos reduciendo el problema en cada llamada recursiva descontando 1 al valor del tope. Recuerde que este parámetro representa la cantidad de elementos con los que estamos trabajando. De esta manera, preguntaremos en cada llamada si el último elemento es = al que estamos buscando, de ser cierto, ya lo encontramos y podremos devolver VERDADERO. En caso contrario, como aún no sabemos si el DATO está o no, debemos consultar en el subarreglo con un elemento menos. En el caso de haber reducido todo el arreglo y quedarnos sin elementos (tope = 0) habremos llegado al caso base, situación en la cual estamos en condiciones de afirmar que elem no está en el arreglo, devolviendo FALSO.

```
/ PRE {tope >= 0}
Funcion Existe (ref x: arreglo[CMAX] de entero, val tope, val elem: entero): lógico
    Si tope > 0 entonces
        Si x[tope] = elem entonces
            existe ← verdadero
        sino
            existe ← existe (x, tope-1, elem)
        fSi
    sino
        existe ← falso
    fSi
fFunc
```

Ilustración 13 función Existe

1.4 MÉTODOS DE ORDENAMIENTO RECURSIVOS

Ahora que ya vimos algunos ejemplos para trabajar con arreglos de manera recursiva, avanzaremos un poco más con los arreglos y veremos dos métodos de ordenamiento.

Existen diversos métodos de ordenamiento de arreglos, entre los más simples se encuentran los métodos denominados Inserción, Selección y Burbujeo. Estos métodos son procesos simples de ordenamiento y se resuelven de manera iterativa. Ahora que ya conoce el proceso recursivo, está en condiciones de estudiar dos métodos más eficientes para ordenar arreglos, que utilizan el proceso Recursivo. Se los presentamos a continuación.

1.4.1 QUICKSORT

El algoritmo de ordenamiento **QuickSort** es un método que se basa en la estrategia divide y vencerás. Está diseñado para ordenar un arreglo o lista de elementos.

Objetivo: Ordenar un arreglo de n elementos en orden ascendente.

Enfoque Principal: Divide el arreglo en segmentos más pequeños y ordena estos segmentos de manera recursiva.

Etapas Principales:

1. El procedimiento quickSort se inicia con el rango completo del arreglo. Se elige un elemento como "pivote" (en nuestro caso tomaremos el último elemento). Los elementos se reorganizan de manera que aquellos menores o iguales al pivote estén a la izquierda y los mayores estén a la derecha.
2. Se divide el arreglo en dos partes: los elementos menores o iguales al pivote y los elementos mayores. Se aplica el algoritmo quickSort recursivamente a ambas partes del arreglo, excluyendo el pivote.
3. Los segmentos más pequeños se vuelven a ordenar utilizando el mismo proceso, y esto continúa hasta que todo el arreglo esté ordenado.

Función de Partición (partición):

Esta función se utiliza para reorganizar los elementos alrededor del pivote. El pivote se coloca en su posición final de manera que los elementos menores estén a la izquierda y los mayores a la derecha. La rutina devuelve la posición final del pivote.

Intercambio de Elementos (intercambiar):

Este procedimiento se utiliza para intercambiar dos elementos en el arreglo.

Ventajas del ordenamiento por Quicksort

Este método de ordenamiento es muy eficiente para arreglos con muchos elementos. Además, es un algoritmo in situ, lo que significa que ordena el arreglo en su lugar sin requerir memoria adicional.

Como un último comentario le podemos decir que es clave en este algoritmo la elección del pivote. Elegir un buen pivote puede afectar el rendimiento del algoritmo. En este ejemplo, para simplificarlo vamos a elegir siempre como pivote al último elemento. Si usted investiga en la bibliografía o en otros autores seguramente encontrará que realizan un proceso para elegir un pivote adecuado, en general suele ser el elemento con un valor cercano al promedio de todos los elementos del arreglo. Como habrá notado, este proceso adicional incorpora un cierto tiempo de procesamiento y puede afectar negativamente a la performance general del ordenamiento.

A continuación, le mostraremos un ejemplo para que vea gráficamente cómo se ordena el arreglo con este método. Luego le presentaremos el algoritmo de Quicksort.

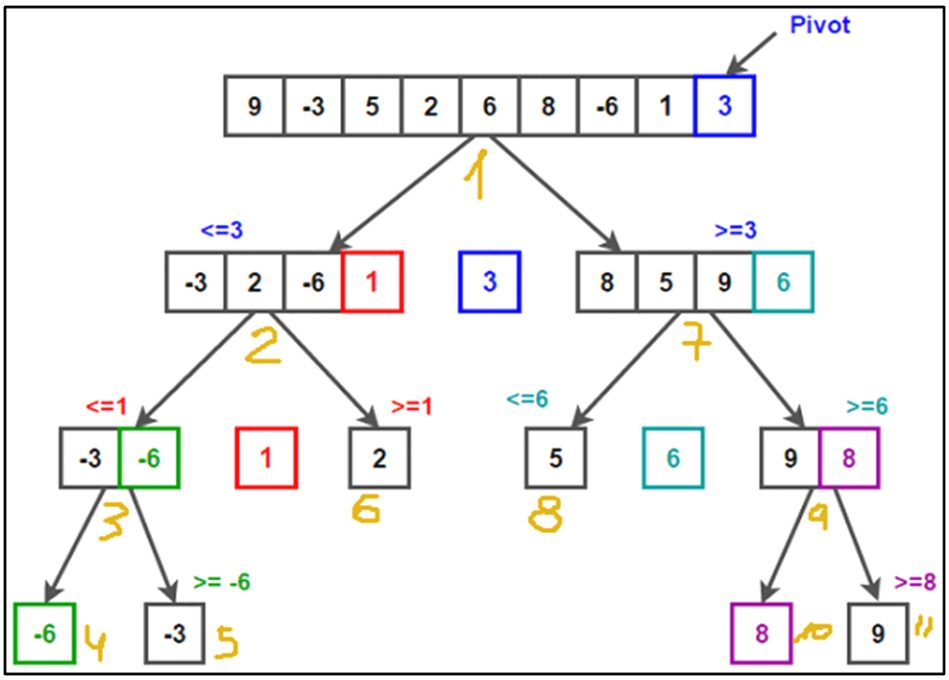

Ilustración 14 Quicksort paso a paso

```
PROCEDIMIENTO quickSort (ref A: TARREGLO, val INICIO, FIN: ENTERO)
VAR LOC
Q: ENTERO
INICIO
    Si (INICIO < FIN) entonces
        Q ← particion(A, INICIO, FIN)
        quickSort (A, INICIO, q-1)
        quickSort (A, q+1, FIN)
    fSi
FIN PROCEDIMIENTO

FUNCIÓN  particion (ref A: TARREGLO, val INICIO, FIN: ENTERO); ENTERO
VAR LOC
I, J, pivot: entero
INICIO
    pivot ← A[FIN]
    I ← INICIO - 1
    para J ← INICIO hasta FIN -1 hacer
        si A[J] <= pivot entonces
            I ← I + 1
            intercambiar(A[I], A[J])
        fSi
    fPara
    intercambiar (A[I +1], A[FIN])
    particion ← I + 1
FIN FUNCIÓN

PROCEDIMIENTO intercambiar (reg A, B : ENTERO)
VAR LOC
    AUX: ENTERO
INICIO
    AUX ← A
    A ← B
    B ← AUX
FIN PROCEDIMIENTO
```

Ilustración 15 Procedimiento Quicksort

1.4.2 MERGESORT

El algoritmo de ordenamiento **MergeSort** es un método diseñado para ordenar un arreglo o lista de elementos.

Objetivo: Ordenar un arreglo de n elementos en orden ascendente.

Enfoque Principal: Divide la lista en segmentos más pequeños, los ordena y luego los combina para obtener una lista ordenada.

Etapas Principales:

1. El procedimiento mergeSort se inicia con el rango completo del arreglo.
2. Se divide el arreglo en dos mitades: izquierda y derecha.
3. Se aplica el algoritmo mergeSort recursivamente a ambas mitades.
4. Finalmente, se fusionan las dos mitades ordenadas utilizando el procedimiento merge.

Función de Mezcla (merge):

Este Procedimiento se encarga de combinar dos segmentos ordenados en un solo segmento ordenado. Utiliza dos arreglos auxiliares, arrIzq y arrDer, para almacenar temporalmente los elementos de las dos mitades. Compara y mezcla los elementos de arrIzq y arrDer en el arreglo original A.

Cómo Funciona:

Este algoritmo divide el arreglo en mitades más pequeñas de manera recursiva hasta que se tengan segmentos de un solo elemento (considerados ordenados por definición).

Luego, combina estos segmentos en uno solo, asegurándose de que los elementos se mantengan ordenados.

Ventajas del ordenamiento por Mergesort

Este método de ordenamiento es eficiente para arreglos con muchos elementos. Su performance es estable y conserva el orden relativo de elementos con claves iguales.

También es importante que observe que este método requiere espacio adicional para los arreglos auxiliares durante el proceso de mezcla. Por lo tanto, si bien el método es rápido, para arreglos muy largos necesitará suficiente espacio en memoria para almacenar los subarreglos auxiliares.

A continuación, le mostraremos un ejemplo para que vea gráficamente cómo se ordena el arreglo con este método. Luego le presentaremos el algoritmo de Mergesort.

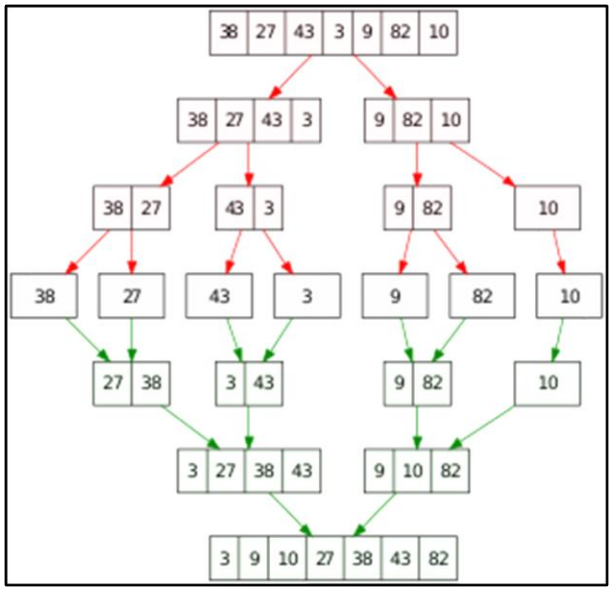

Ilustración 16 Mergesort paso a paso

```
PROCEDIMIENTO mergeSort (ref A: TARREGLO, val izq, der: ENTERO)
   VAR LOC: med: entero
INICIO
    Si (izq < der) entonces
        med ← (izq + der) DIV 2
        mergeSort (A, izq, med)
        mergeSort (A, med+1, der)
        merge(A, izq, med, der)
    fSi
FIN PROCED


PROCEDIMIENTO merge (ref A: TARREGLO, val izq, med, der: ENTERO)
VAR LOC
    I, J, K, aux1, aux2: entero
    arrIzq: arreglo de aux1 enteros     // creando arrays auxiliares arrIzq y arrDer
    arrDer: arreglo de aux2 enteros

INICIO
    aux1 ← med – izq + 1
    aux2 ← der – med


    Para I desde 1 hasta aux1 hacer  //copiando datos a arrays auxiliares
        arrIzq[I] ← A[izq + I - 1]
    fPara
    Para J desde 1 hasta aux2 hacer
        arrDer[J] ← A[med + J]
    fPara

    I ← 1           //uniendo los datos de arrays temporales de vuelta en A[izq...der]
    J ← 1
    K ← izq
    Mientras (I <= aux1) y (J <= aux2) hacer
        Si  arrIzq[I] <= arrDer[J] entonces
            A[K] ← arrIzq[I]
            I ← I + 1
        sino
            A[K] ← arrDer[J]
            J ← J + 1
        fSi
        K ← K + 1
    fMientras

    Mientras I <= aux1 hacer     //copiando elementos remanentes de arrIzq, si es que quedan
        A[K] ← arrIzq[I]
        I ← I + 1
        k ← k + 1
    fMientras
    Mientras J <= aux2 hacer    //copiando elementos remanentes de arrDer, si es que quedan
        A[K] ← arrDer[J]
        J ← J + 1
        K ← K + 1
    fMientras
FIN PROCED
```

Ilustración 17 Procedimiento Mergesort

3- CONCLUSIONES

En esta unidad le presentamos una nueva manera de resolver algoritmos, llamada Recursividad. Este método se caracteriza por dividir el problema en subproblemas más pequeños, hasta encontrar el caso base en el que se pueda resolver. Esta forma de dividir el problema se realiza desde el algoritmo mediante una rutina que se invoca a sí misma, de modo recursivo. Durante esta Unidad también comparamos este proceso Recursivo con la forma que ya conoce, el proceso Iterativo. Finalmente, le presentamos un ejemplo recursivo analizando la función Factorial, y analizamos dos métodos de ordenamientos de arreglos recursivos, llamados Quicksort y Mergesort.

4- BIBLIOGRAFÍA DE REFERENCIA

Bisbal Riera, J. (2013). Manual de algorítmica: recursividad, complejidad y diseño de algoritmos. Editorial UOC.
https://elibro.net/es/ereader/caece/56561?page=50

Sedgewick, R. y Wayne, K. (2011). Algorithms. 4th Edition. Addison- Wesley.

Cormen, T., Leiserson, C., Rivest, R. & Stein, C. (2022) Introduction to algorithms. 4th Edition. The MIT Press.